

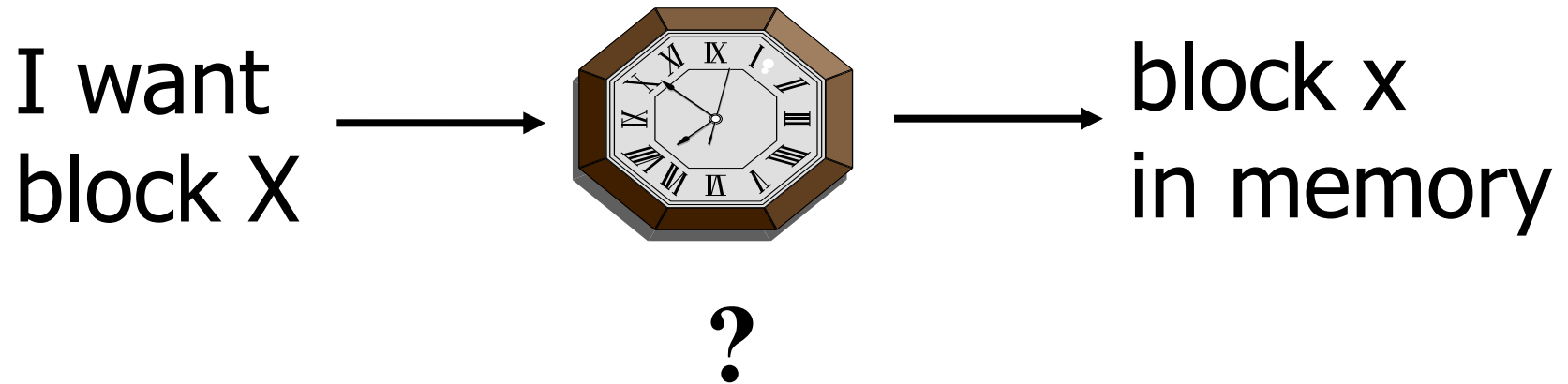
CS4432: Database Systems II

Record Representation

Updates

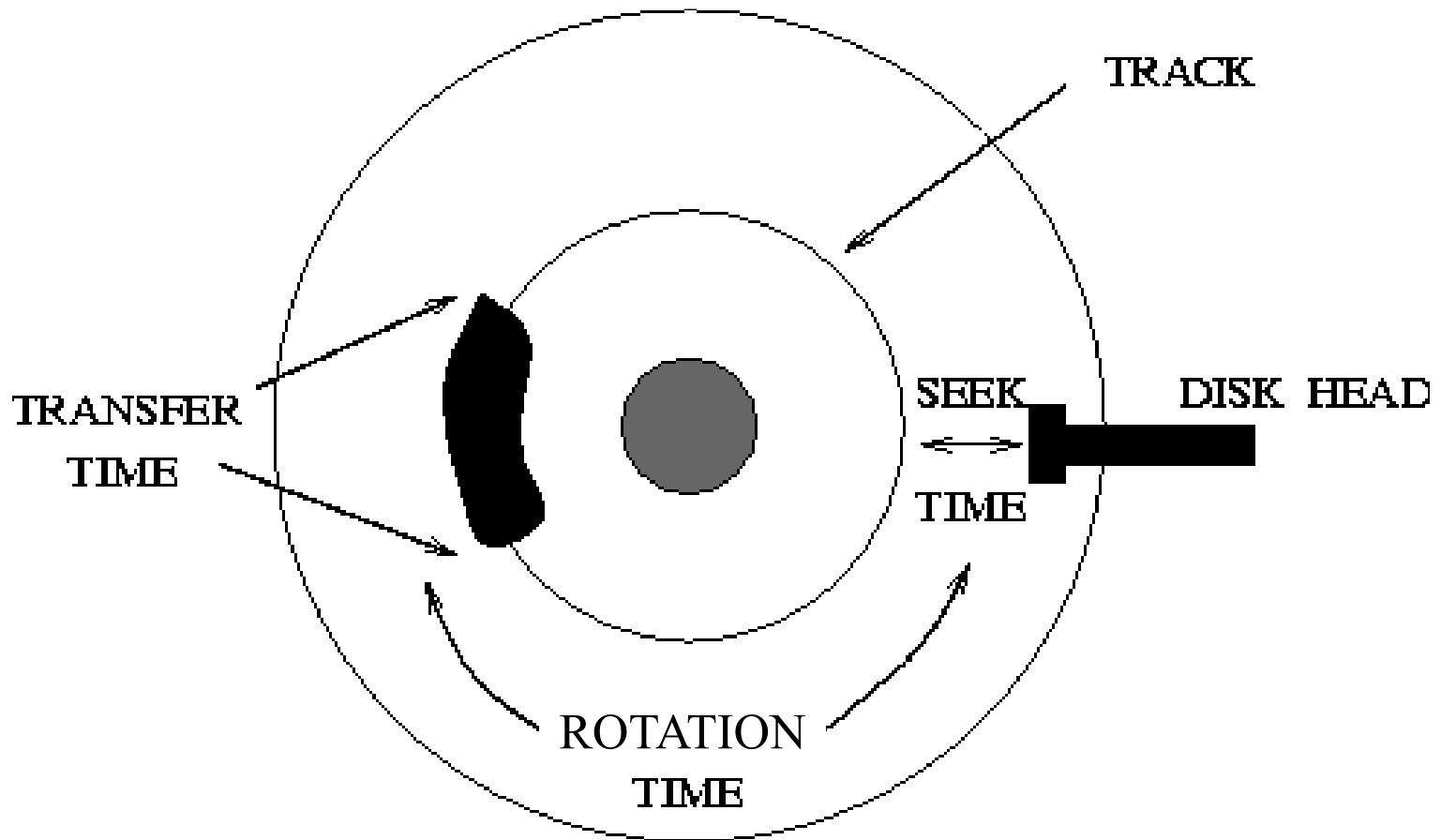
- **Quiz 1 – Today -> Must be in the classroom**
- **HW 1** – due on **Thursday** (Mar 26)
- **Project 1** – to be released on the same **Thursday** (topic covered over the week)
- Please come to the office hours!

What about access time?



Latency = Disk Delay {seek & rotation} +
Transfer Time

Recap: Seek, Rotation & Transfer Time



Accelerating Access to Blocks

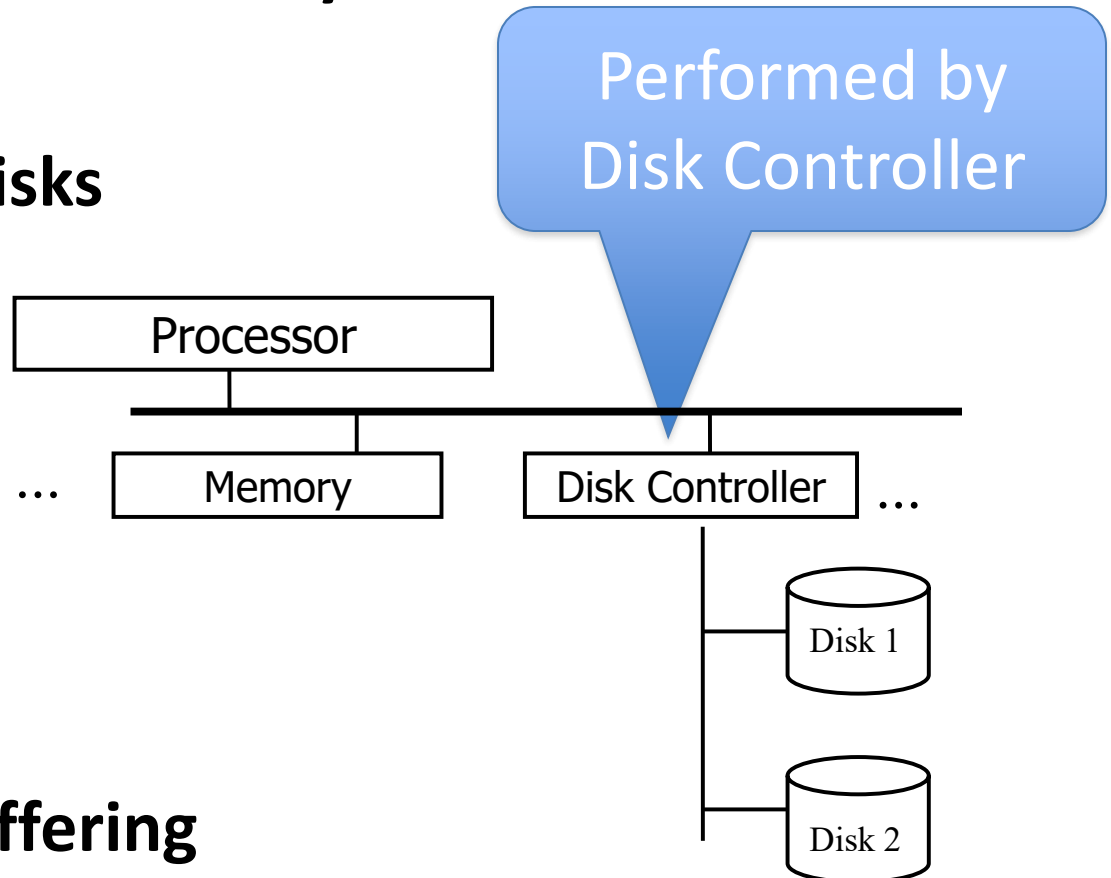
1. Placing Related Blocks on Cylinders

2. Using Multiple Disks

3. Mirroring

4. Disk Scheduling

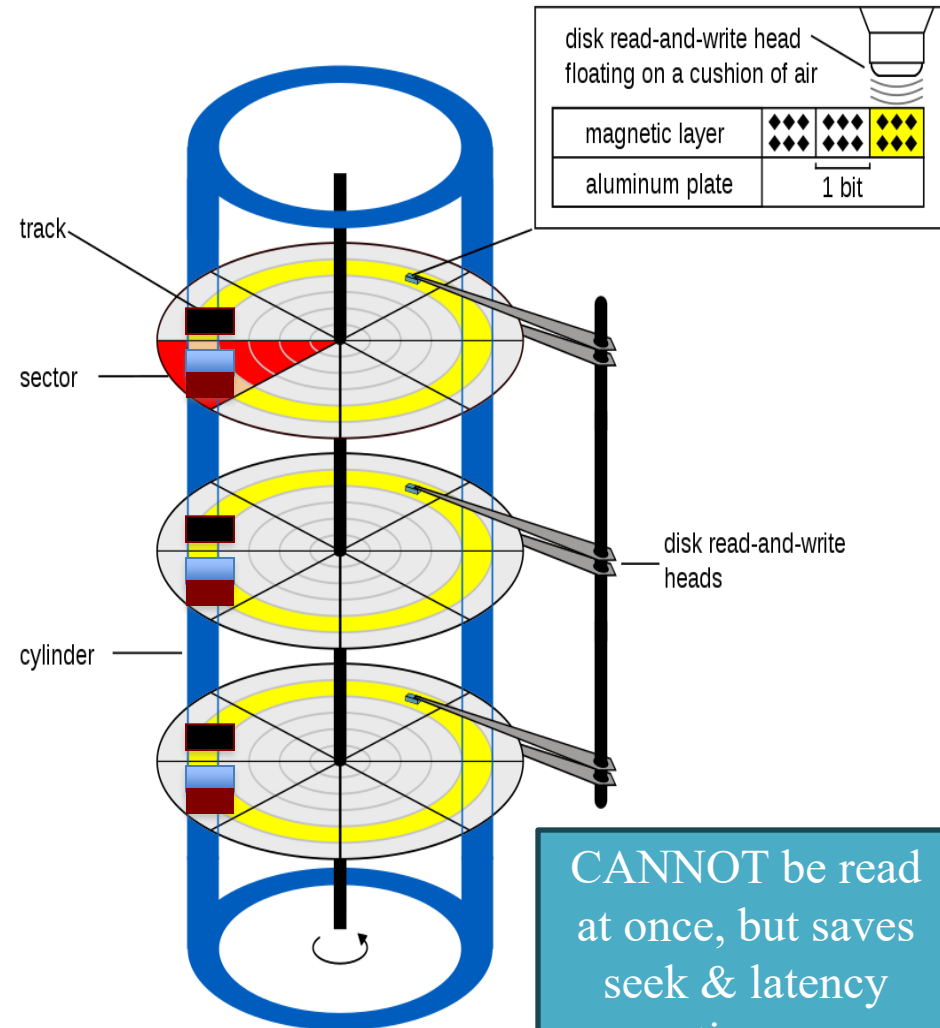
5. Prefetching & Buffering



#1: Placing Related Blocks on Cylinders

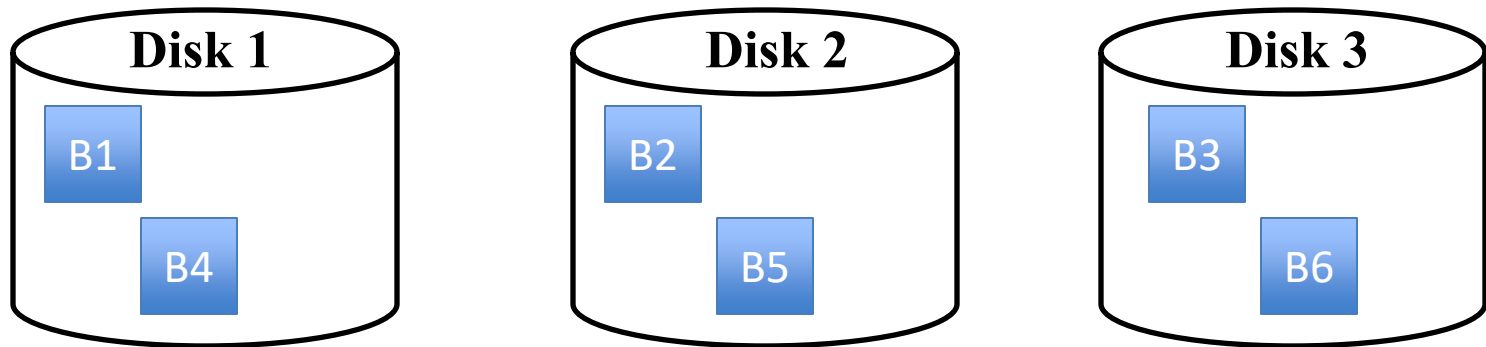
- Start by adding blocks on the next sectors on the same track
- Put on the same cylinder to save seek & rotation times
- Make use of the 2 surfaces in the same cylinder

- Put on the next sector on same track
- Put on the top surface in same cylinder
- Put on the bottom surface in same Cylinder as the blue ones



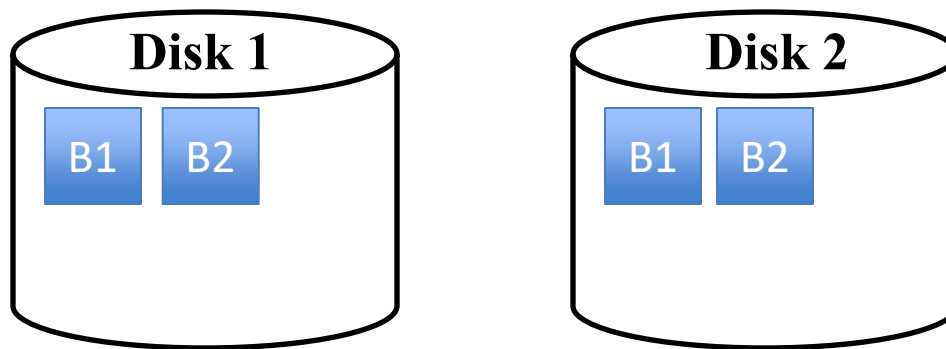
2- Using Multiple Disks: Striping

- Use multiple smaller disks instead of one large disk
- Each disk can access its data independently
 - N disks → N times faster access



3- Mirroring

- Use pairs of disks that are mirrors to each other
- Good for failure & Good for faster access
- Higher overhead under writing operations



4- Disk Scheduling: SCAN

- **Input**
 - Current position of the head (track number)
 - Direction: left (outer), or right (inner)
 - queue of requests
- **Algorithm**
 - Move in the decided direction
 - Serve requests in the order of seeing them
 - Reach the end of the disk in this direction
 - Reverse direction
 - Serve remaining requests in the order seeing them
- **What 'seeing' means?**
 - If request for track X arrived by the time the head reaches X, **it is served**

5- Prefetching & Buffering

- If DBMS can predict the sequence of access
 - It can pre-fetch and buffer more blocks even before requesting them.

Example: **Have a File**

» Sequence of Blocks B1, B2, ...

Have a Program

» Process B1

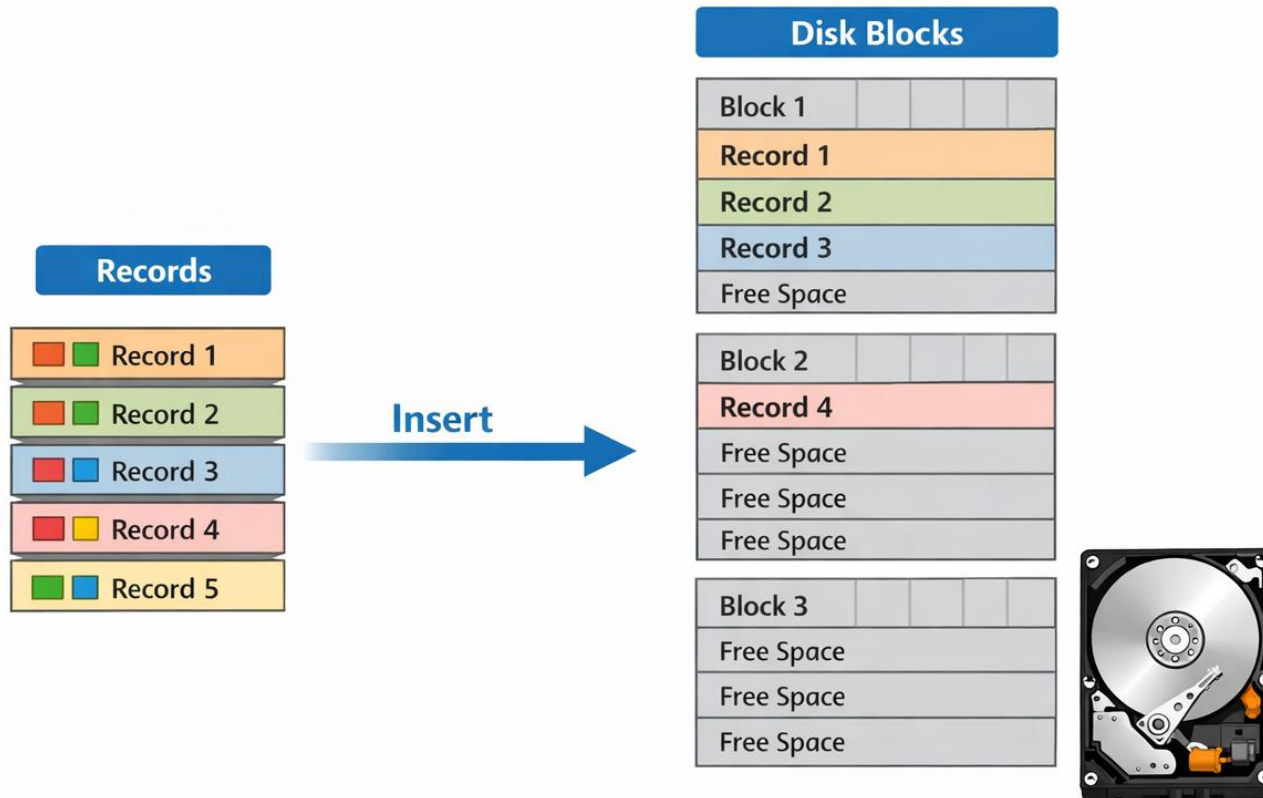
» Process B2

» Process B3

...

Learning objectives

- Understand the structure of fixed- and variable-length records
- Explain how to access fields in a record
- Compute how many records can fit in a disk block



HOW MANY DISK BLOCKS ARE NEEDED FOR STORING RECORDS OF A TABLE?

Database Records (tuples) Fields

First Name	Last Name	Address	City	Age
Mickey	Mouse	123 Fantasy Way	Anaheim	73
Bat	Man	321 Cavern Ave	Gotham	54
Wonder	Woman	987 Truth Way	Paradise	39
Donald	Duck	555 Quack Street	Mallard	65
Bugs	Bunny	567 Carrot Street	Rascal	58
Wiley	Coyote	999 Acme Way	Canyon	61
Cat	Woman	234 Purrfect Street	Hairball	32
Tweety	Bird	543	Itotiltaw	28

- Each **record**, and each **field** is just a sequence of bytes on a disk block
- Field has **type** that can be **fixed-** or **variable-length**

How Records are Stored on Disk

Two types of records

Fixed-Length Record

All records have
the same size

First Name	Last Name	Address	City	Age
Mickey	Mouse	123 Fantasy Way	Anaheim	73
Bat	Man	321 Cavern Ave	Gotham	54
Wonder	Woman	987 Truth Way	Paradise	39
Donald	Duck	555 Quack Street	Mallard	65
Bugs	Bunny	567 Carrot Street	Rascal	58
Wiley	Coyote	999 Acme Way	Canyon	61
Cat	Woman	234 Purrfect Street	Hairball	32
Tweety	Bird	543	Itotitaw	28

Variable-Length Record

Different records may
have different sizes

Check the record's fields

If all fixed size → Fixed-Length record

if any field is variable size → Variable-Length record

Fixed-Length Record Example

Create Table star (

ID Int,

Name char(30),

Address char(255),

Gender char(1),

DOB Date)

4 bytes

30 bytes

255 bytes

1 byte

10 bytes

ID	Name	Address	Gender	DOB
----	------	---------	--------	-----

Variable-Length Record Example

```
Create Table star (  
  ID Int,  
  Name varchar2 (30),  
  Address varchar2(255),  
  Gender char(1),  
  DOB Date)
```

Variable length and at
most 255 bytes

ID	Name...	Address...	Gender	DOB
----	---------	------------	--------	-----



Data Types

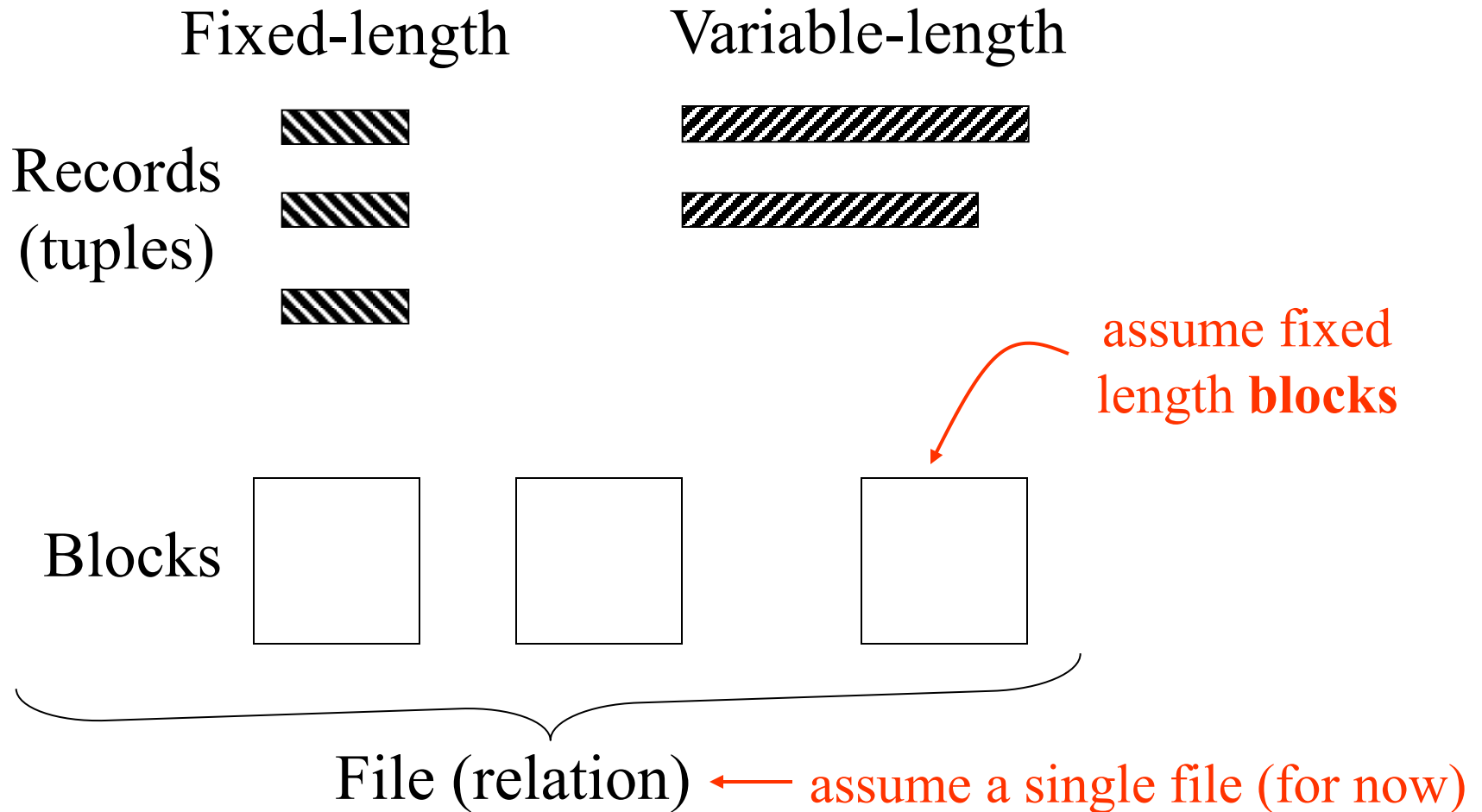
- Know which ones are fixed size vs. variable size

MySQL DATATYPES

DATE TYPE	SPEC	DATA TYPE	SPEC
CHAR	String (0 - 255)	INT	Integer (-2147483648 to 2147483647)
VARCHAR	String (0 - 255)	BIGINT	Integer (-9223372036854775808 to 9223372036854775807)
TINYTEXT	String (0 - 255)	FLOAT	Decimal (precise to 23 digits)
TEXT	String (0 - 65535)	DOUBLE	Decimal (24 to 53 digits)
BLOB	String (0 - 65535)	DECIMAL	"DOUBLE" stored as string
MEDIUMTEXT	String (0 - 16777215)	DATE	YYYY-MM-DD
MEDIUMBLOB	String (0 - 16777215)	DATETIME	YYYY-MM-DD HH:MM:SS
LONGTEXT	String (0 - 4294967295)	TIMESTAMP	YYYYMMDDHHMMSS
LOBLOB	String (0 - 4294967295)	TIME	HH:MM:SS
TINYINT	Integer (-128 to 127)	ENUM	One of preset options
SMALLINT	Integer (-32768 to 32767)	SET	Selection of preset options
MEDIUMINT	Integer (-8388608 to 8388607)	BOOLEAN	TINYINT(1)

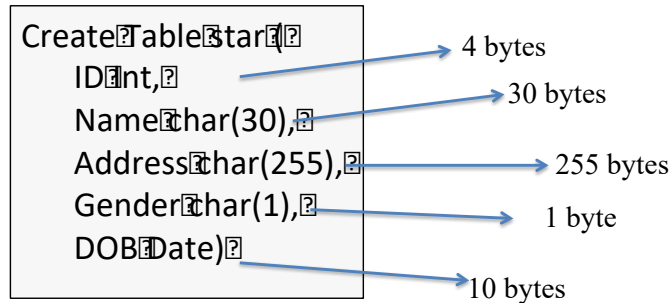
Copyright © mysqltutorial.org. All rights reserved.

Placing Records in Disk Blocks



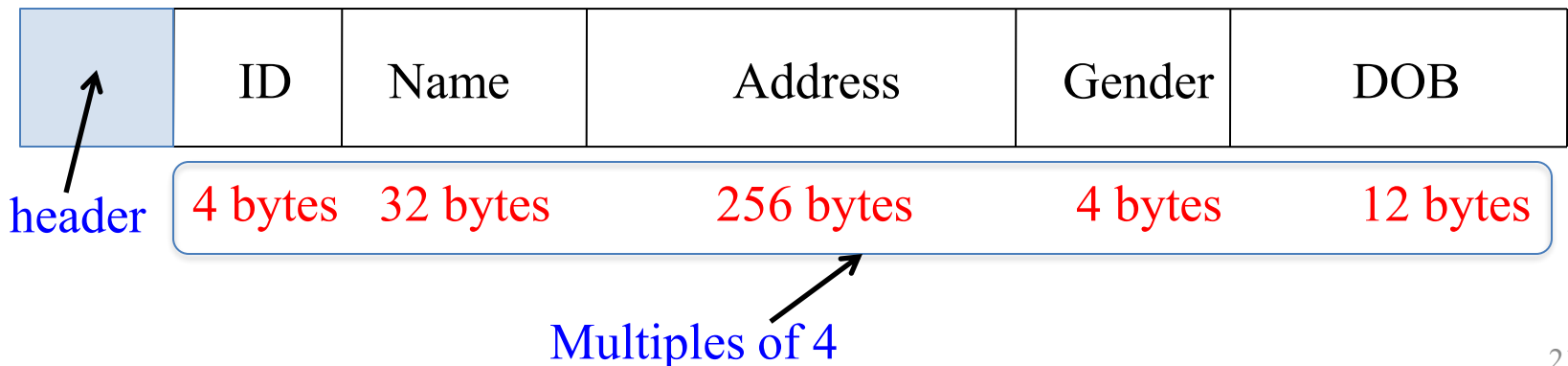
Fixed-Length Records

Representing Tuples (Fixed-Length)



1- All fields are **aligned** to start at 4- or 8-byte boundaries (depends on HW & OS requirements) & **concatenated**

2- Each record has a **header** holding some info



Record Header (Fixed-Length)

- Often it is convenient to keep some "header" information in each record:
 - A pointer to schema information (attributes/fields, types, their order in the tuple, constraints)
 - Length of the record/tuple
 - Timestamp of last modification

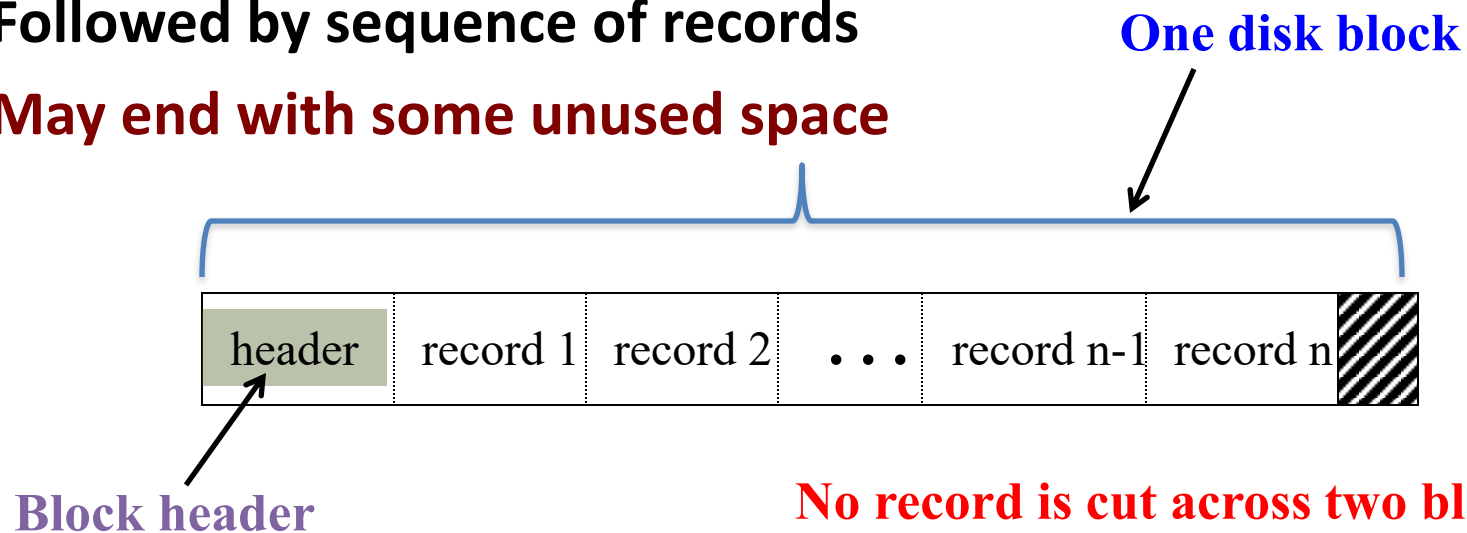


Packing Records into Blocks (Fixed-Length)

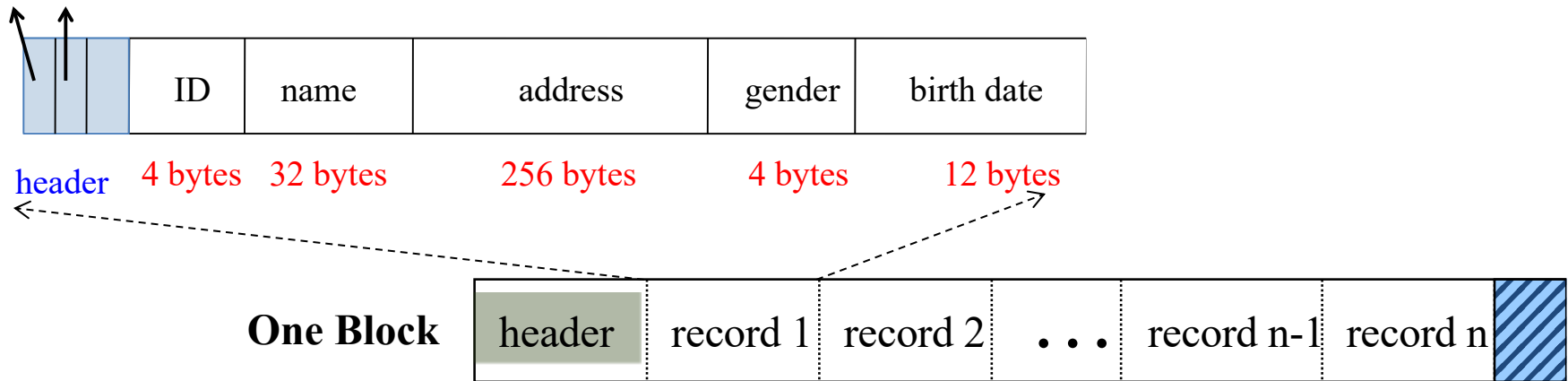
- Start with a **block header**:
 - Timestamp of last modification/access
 - Links to next and previous blocks in the big file
 - *Info about the records' offsets !!!*
- Followed by sequence of records
- May end with some unused space

First Name	Last Name	Address	City	Age
Mickey	Mouse	123 Fantasy Way	Anaheim	73
Bat	Man	321 Cavern Ave	Gotham	54
Wonder	Woman	987 Truth Way	Paradise	39
Donald	Duck	555 Quack Street	Mallard	65
Bugs	Bunny	567 Carrot Street	Rascal	58
Wiley	Coyote	999 Acme Way	Canyon	61
Cat	Woman	234 Purrfect Street	Hairball	32
Tweety	Bird	543	Itottaw	28

Records



Access in Fixed-Length Records



- Information about field types are same for all records in a file; stored in *system catalogs*.
- Finding i 'th record in a block does not require scan over previous records
- Finding i 'th field does not require scan over previous fields

Important

The block must be brought to memory first

Variable-Length Records

Variable Length ~~Records~~ Fields

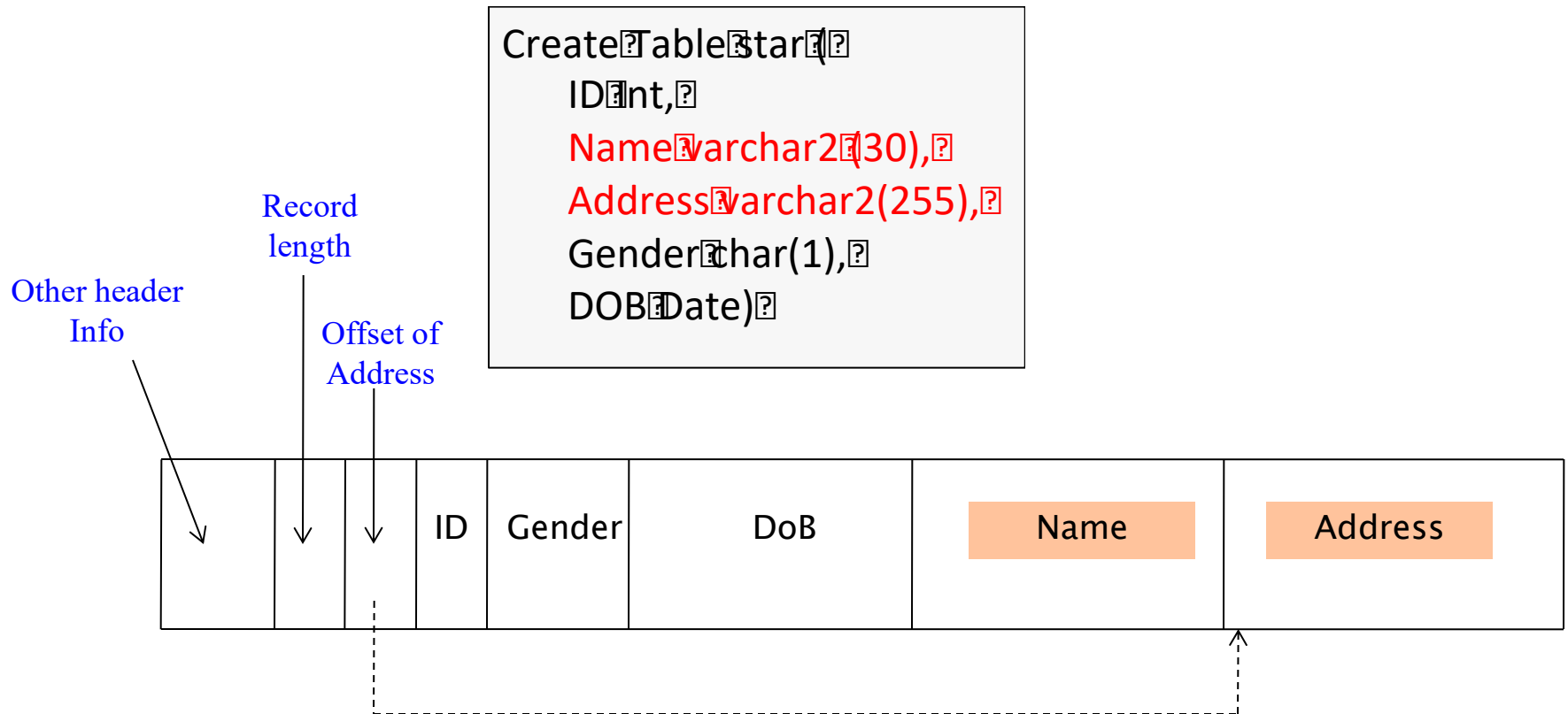
- A **field** whose length can be different across records
- Data items with varying size (e.g., if maximum size of a field is large but most of the time the values are small)
- Complicates design and access, but sometimes efficient to have
- **E.g., comment field in news media... most comments are short, but few are very long → varchar(1000)**

Records with Variable Fields

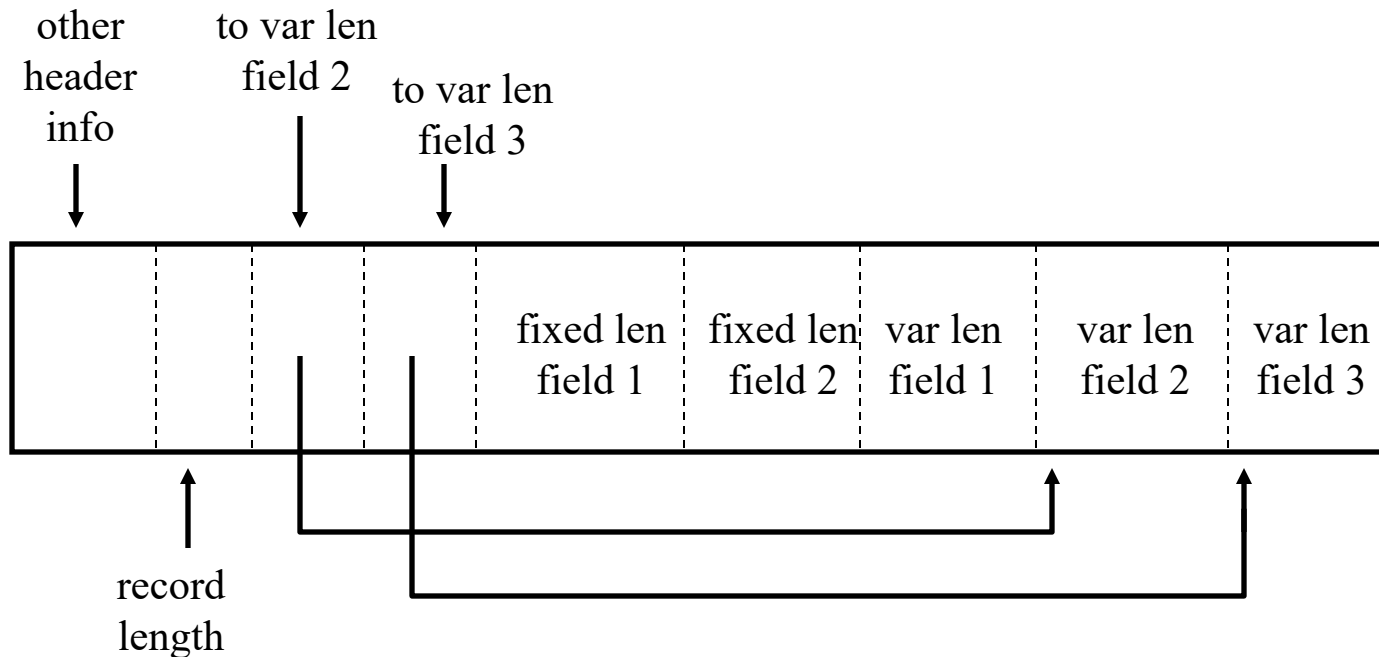
Effective way to represent variable length records:

- Fixed length fields are kept ahead of the var. length fields
- **Record header contains**
 - Length of the record
 - Pointers to beginning of **each** variable length **field** except the first one.

Records with Variable-Length Fields



Extend to Multiple Fields

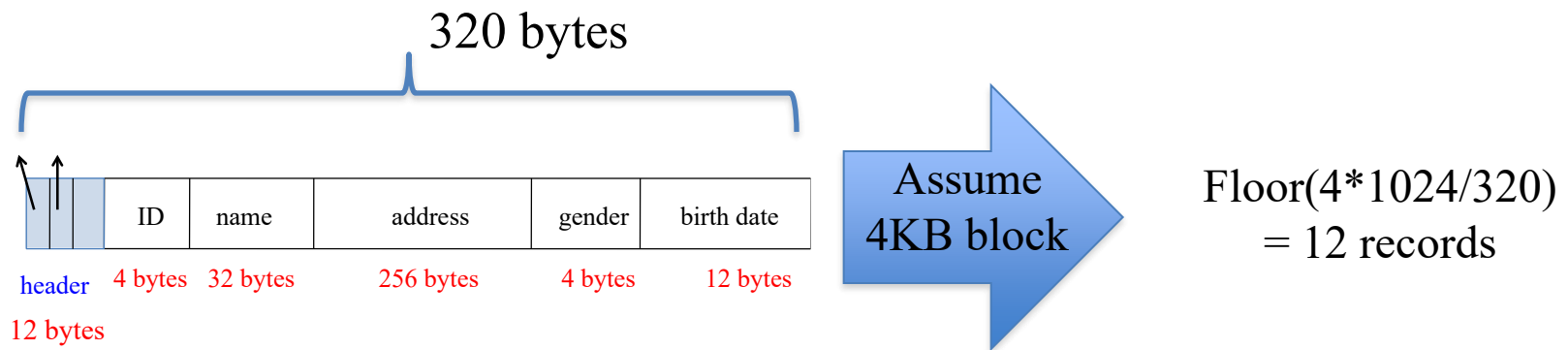


**Efficient access*

- Still reading the i^{th} field, does not require scanning over previous fields

Question: How many records can fit in a disk Block

- We can do the math for fixed-length records



- For variable-length records, we can only estimate a range

Summary

- **Two types of records**
 - **Fixed Length**
 - **Variable Length**
- **The type depends on the data types of the fields**
- **Calculating the size of a record under 4- or 8-byte boundaries**
- **Record representation as bytes**

CS4432: Database Systems II

Packing Records into Blocks

Learning objectives

- Recognize limitations with **packed** and **bitmap** approaches for storing records
- Be able to implement these operations on variable-length slot directory approach:
 - Insertion
 - Update
 - Deletion

Packing Approaches

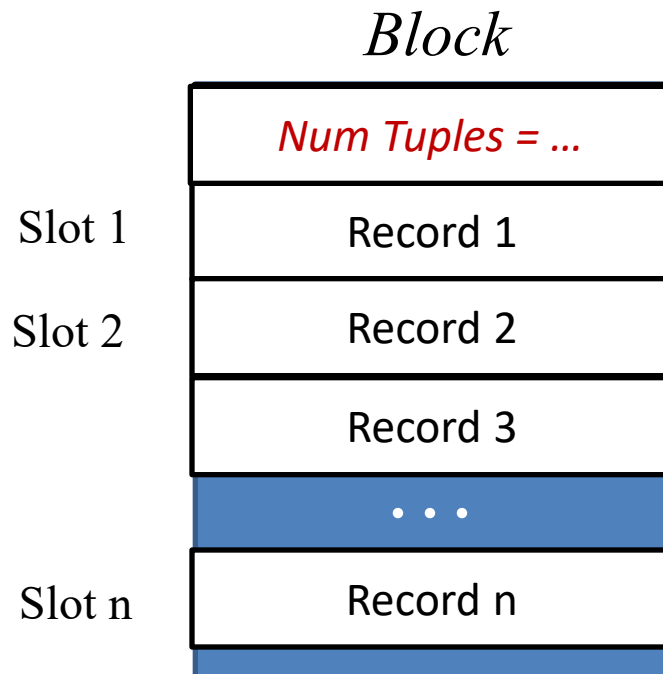
- **Packed Approach**
- **BitMap/UnPacked Approach**
- **Variable-Length Slot Directory Approach**

How could we store records in a block?



Assume
fixed-length
for now

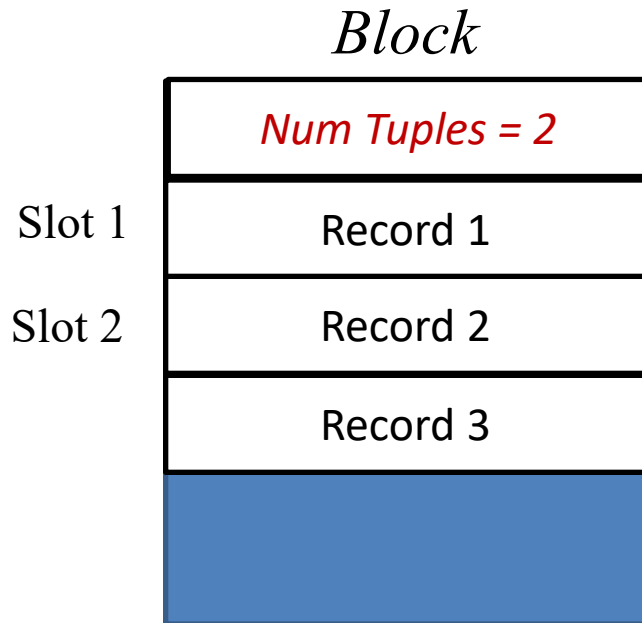
Alternative visualization:



What if the
block is
NOT full?

Block Format : Fixed-Length Records

Packed Approach



Packed means **no “holes”**

Insertion

- If enough free space (at the end) then insert in this block
- Increment Num_Tuples

WARNING!

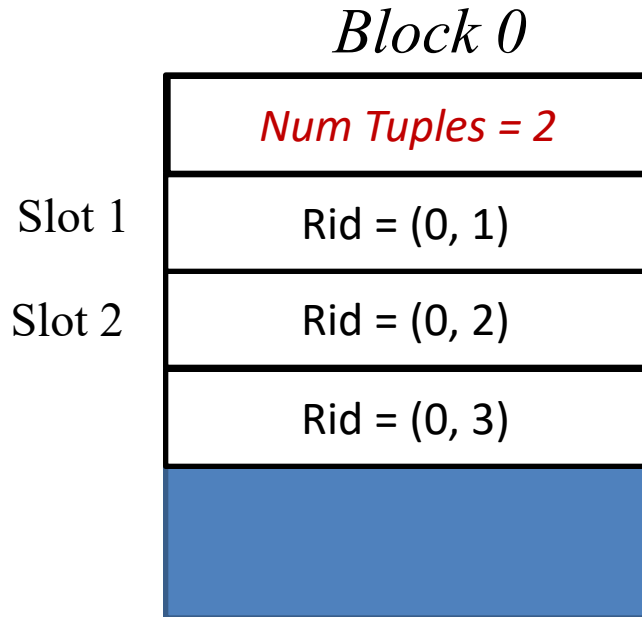
What happens if we delete a tuple?

- Move the last record to fill in the empty space
- Decrement N

Moving records
would require
copying data &
changing headers

Block Format : Fixed-Length Records

Packed Approach



Record header contains:

Record id (rid) = $\langle \text{Block id, slot \#} \rangle$.

Physical
Address

Logical
Address

- In this approach, moving records for free space management $\rightarrow$ changes Record id

Goal:
Keep rid as is

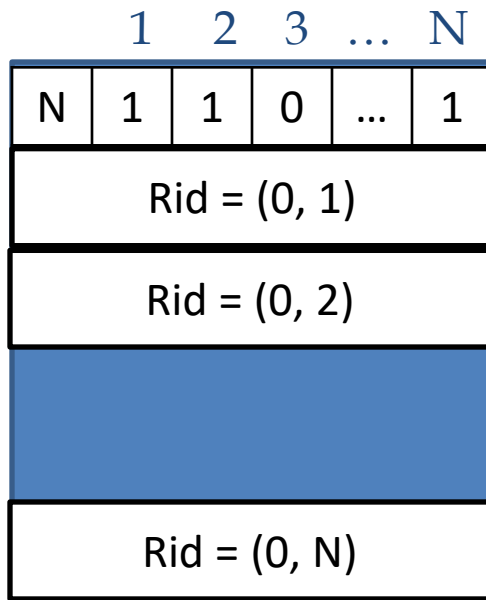
Usually not acceptable to change the Record id

Block Format : Fixed-Length Records

BitMap/UnPACKED Approach

number
of slots

UNPACKED, BITMAP



□ Every slot in the block has a bit (0 or 1)

Insertion

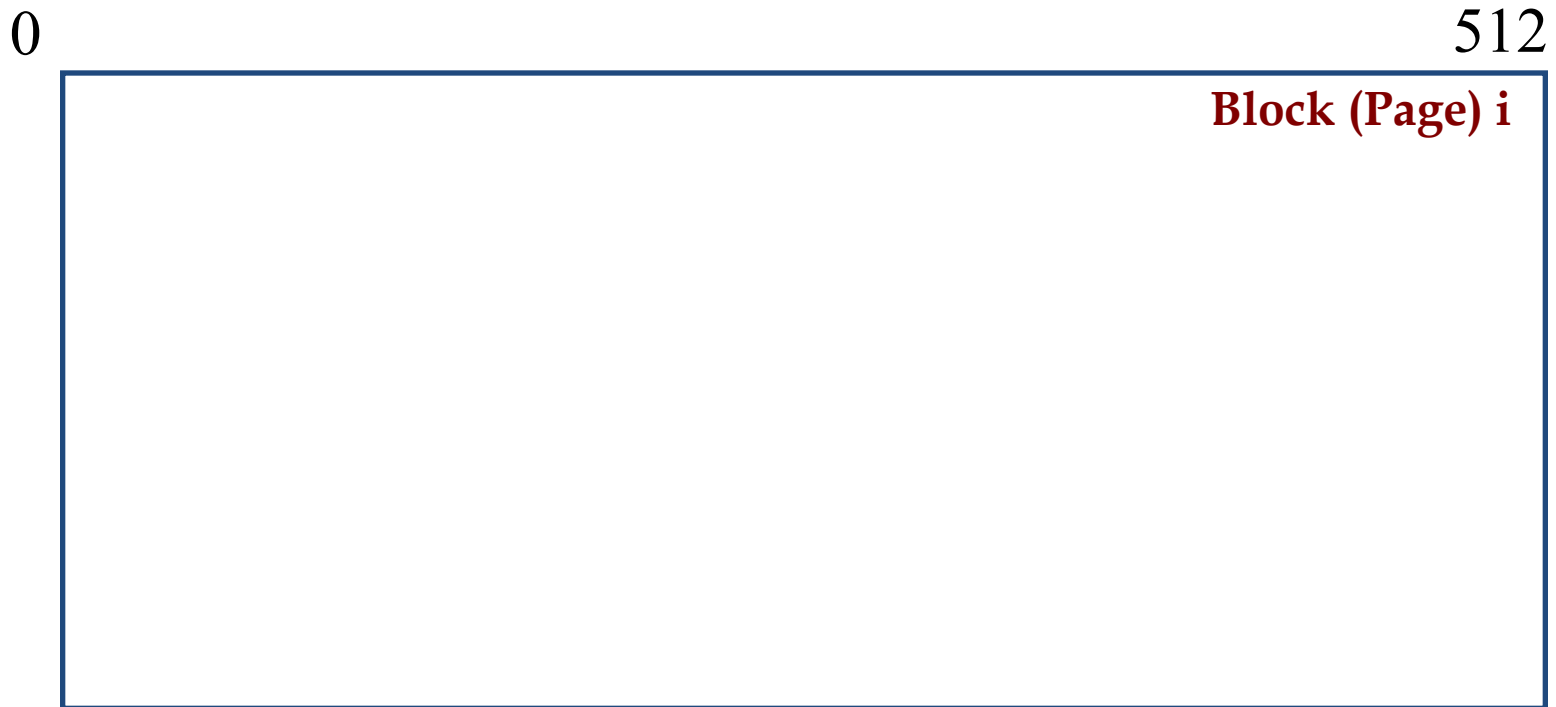
- Find free slot anywhere in the block
- Insert the record (increment N)
- Set its bit to 1

Deletion (No movement)

- Decrement N
- Set its bit to 0

That is a better approach, but does not work for variable-length records

Block Formats: Variable-Length Records (Scratch slide)



How can we insert
these records?

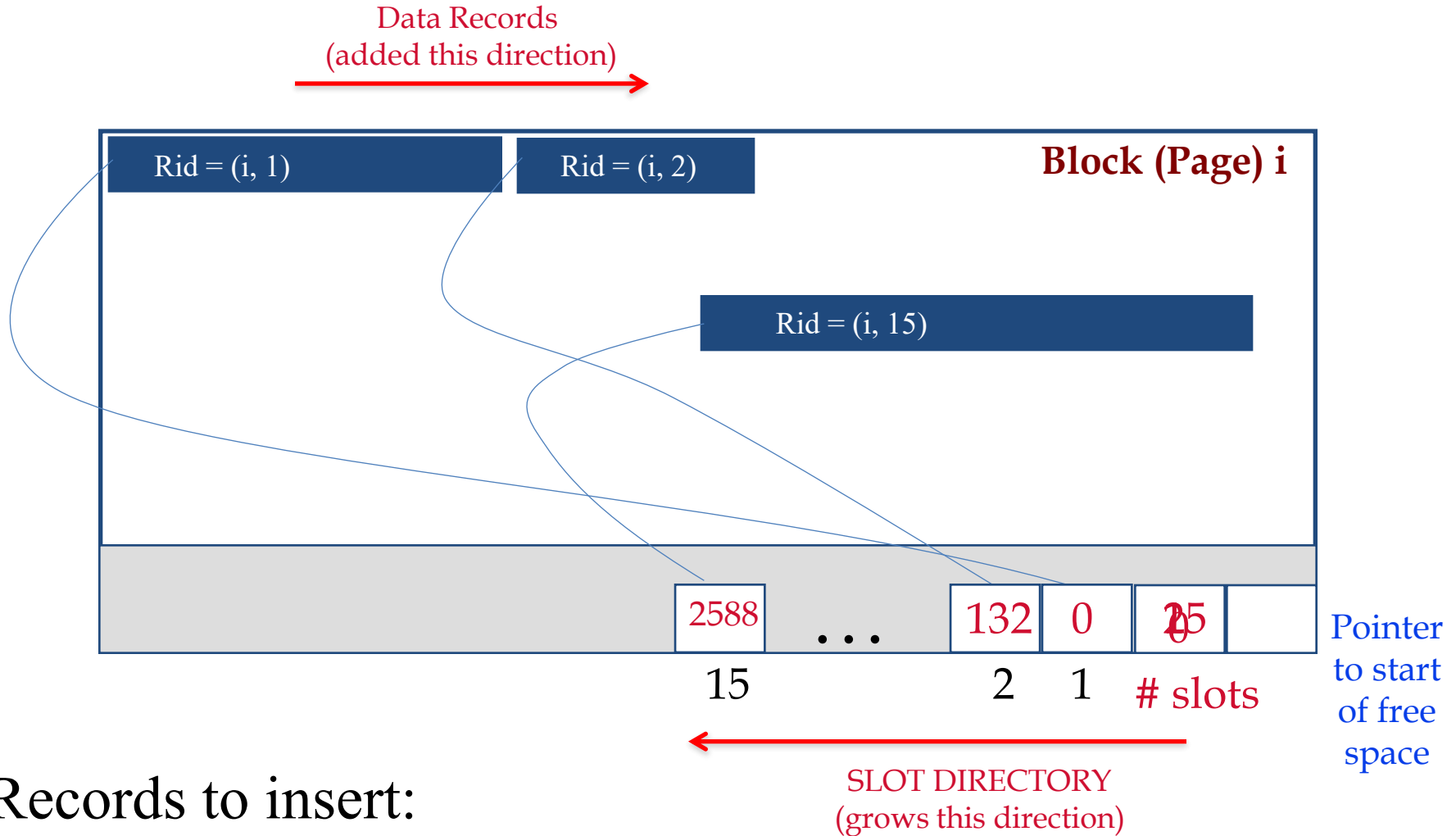
Rid = (i, 1)

132 bytes

Rid = (i, 2)

72 bytes

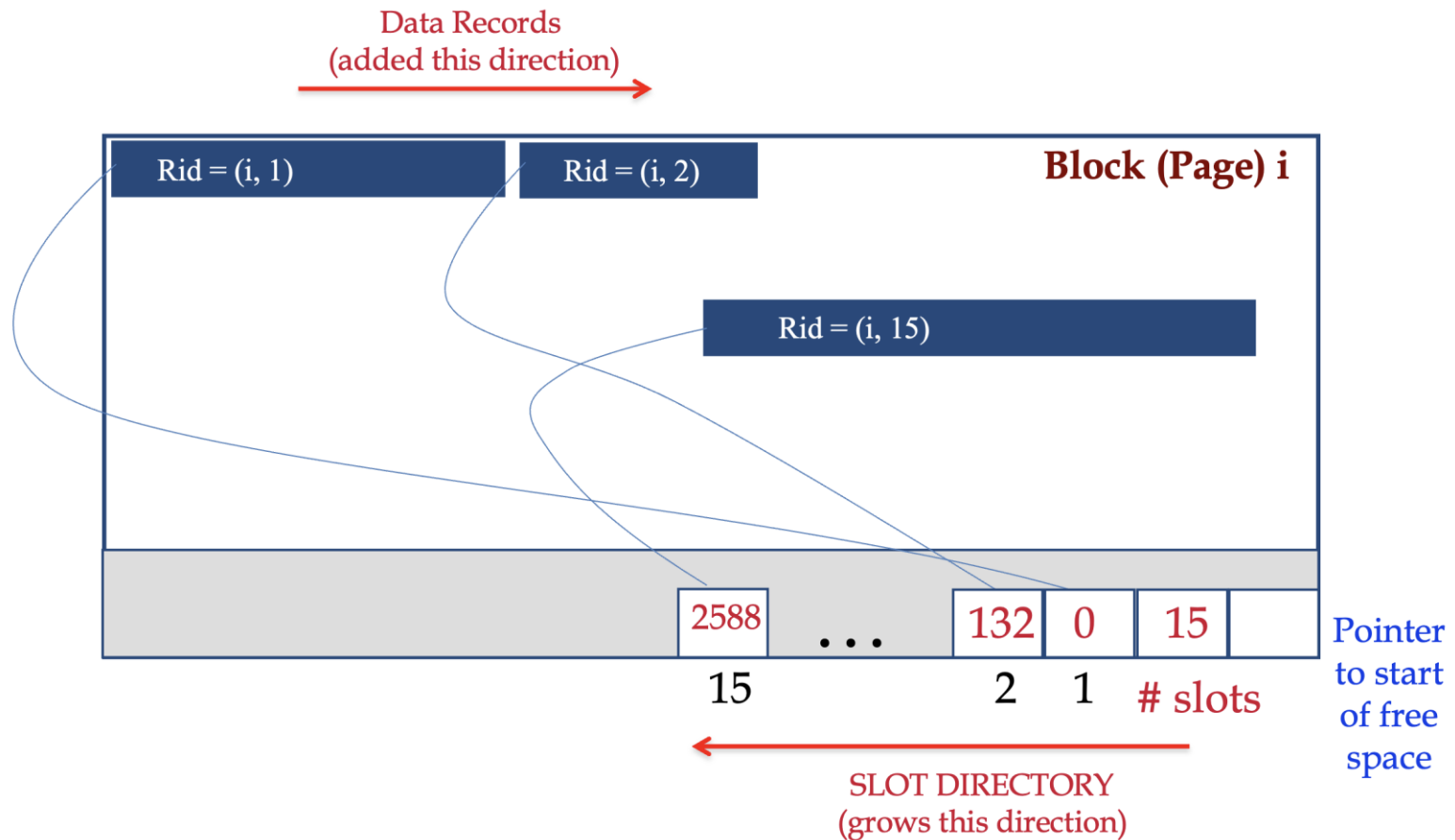
Block Formats: Variable-Length Records



Records to insert:

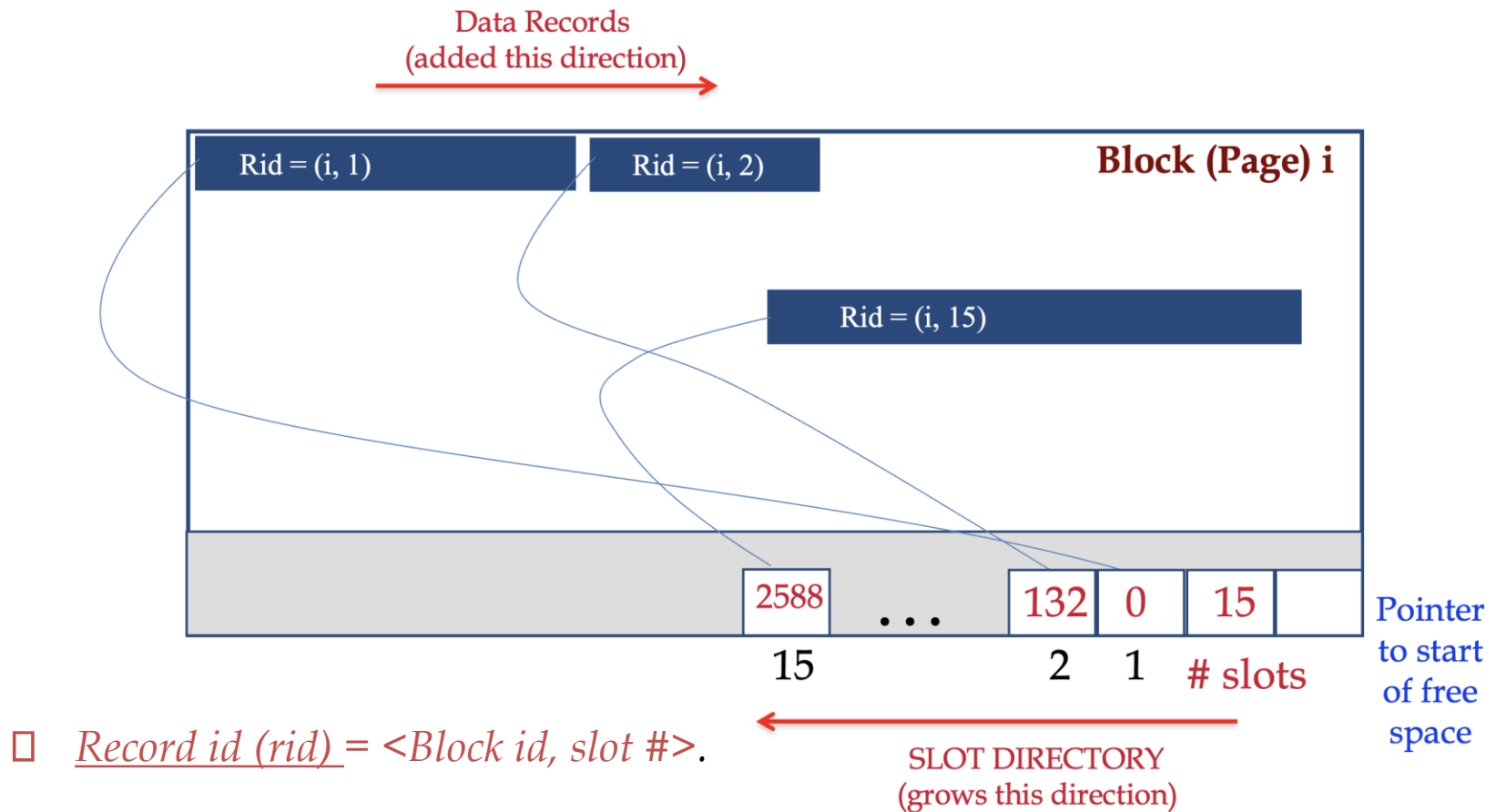
Rid = (i, 1)	132 bytes
Rid = (i, 2)	72 bytes

Block Formats: Variable-Length Records



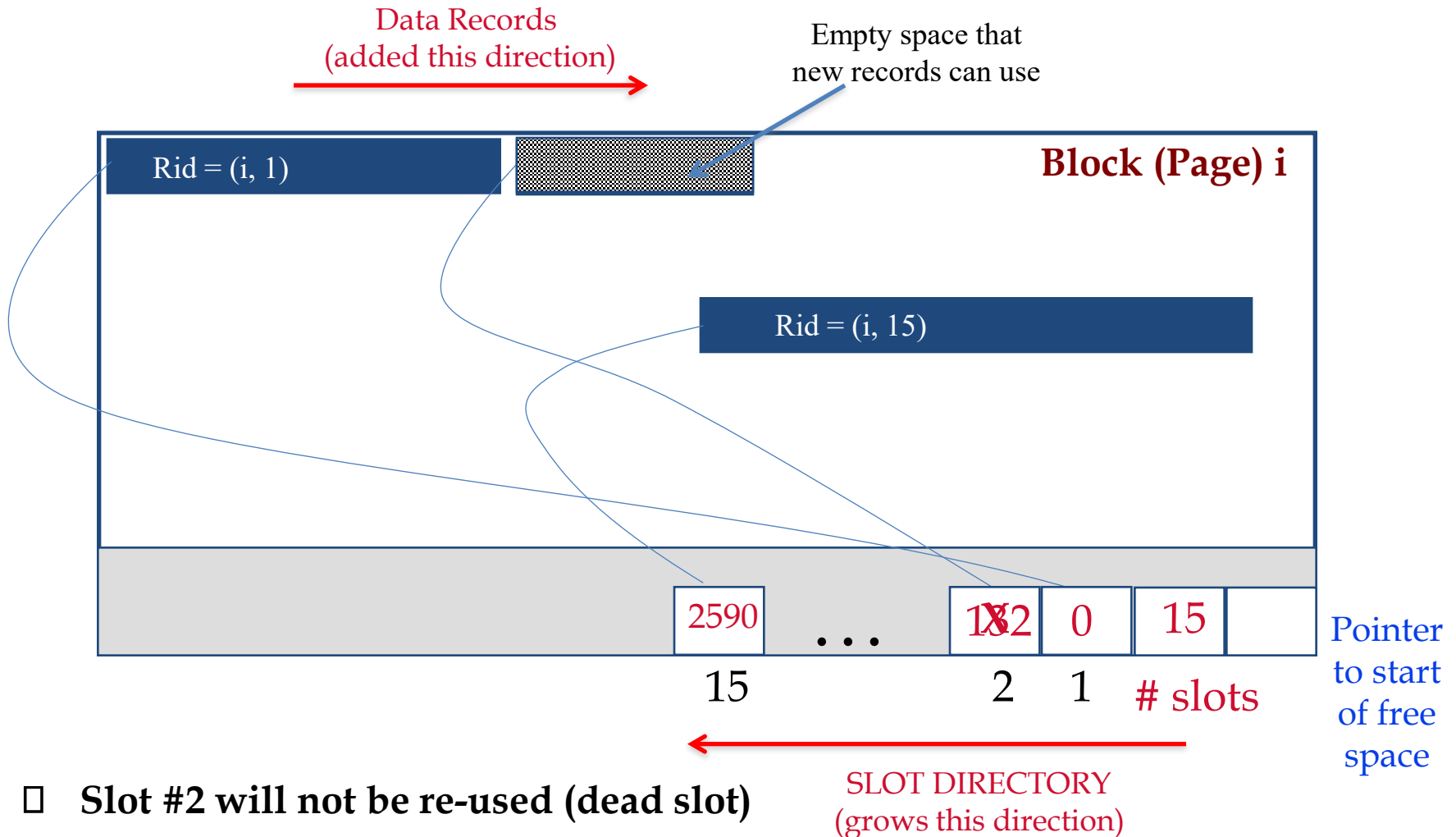
- The slot directory starts from one end and grows as needed
- The data records start from the other end (*No need for fixed allocation for SLOT directory*)

Block Formats: Variable-Length Records



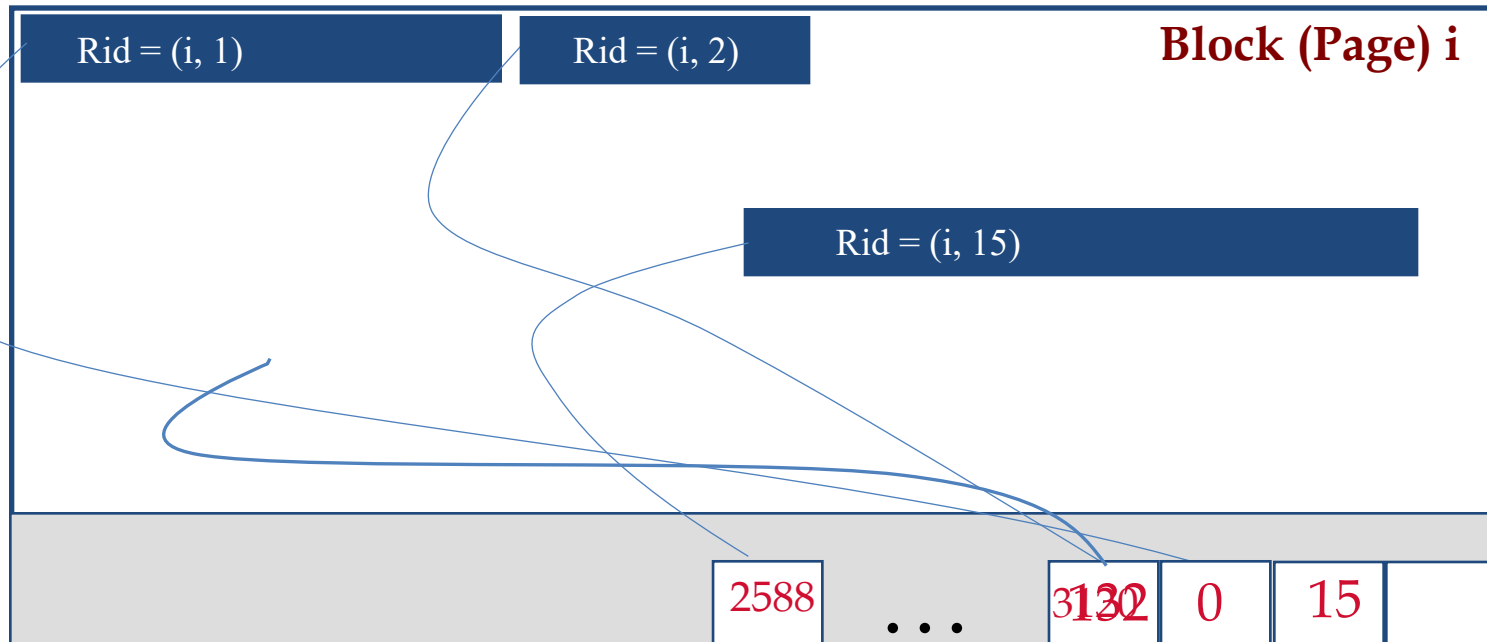
- Rid does not change even if the record moves within the block
- Deleting a record will leave some empty space in the middle

Example: Deleting a record



Example: Moving a record

Data Records
(added this direction)



15

...

3132

0

15

2

1

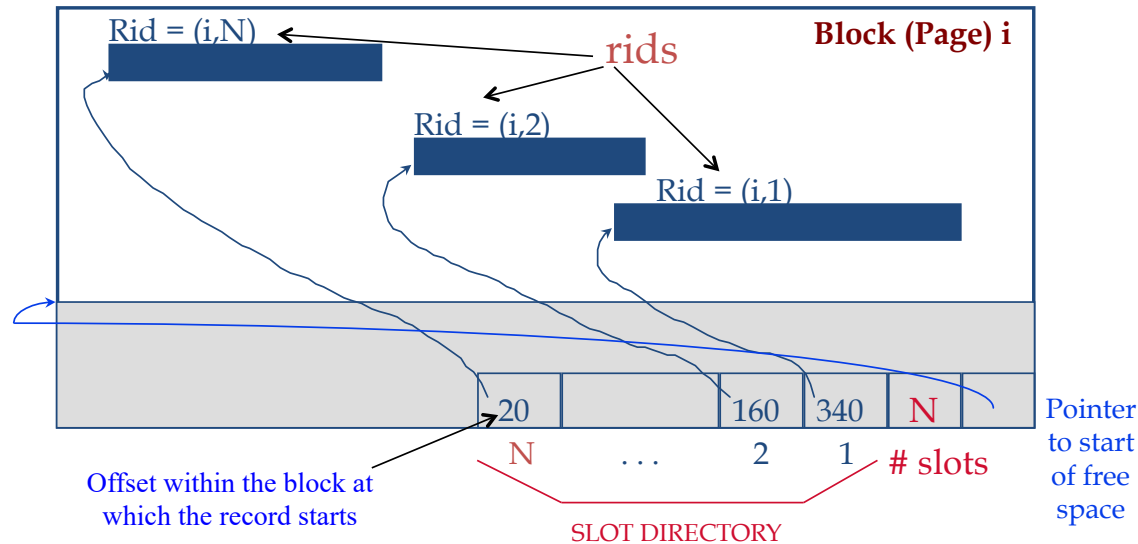
slots



SLOT DIRECTORY
(grows this direction)

Pointer
to start
of free
space

Indirection: Physical vs. Logical Addresses



- This approach of addressing the records combines physical and logical addresses

$$\square \text{ Record id (rid) } = \langle \text{Block id, slot \#} \rangle.$$

Physical Address (which disk,
platter, track and sector)

Logical
Address

Live Demo

- Using OneCompiler's PostgreSQL to code [43bq7vf6e - PostgreSQL - OneCompiler](#)

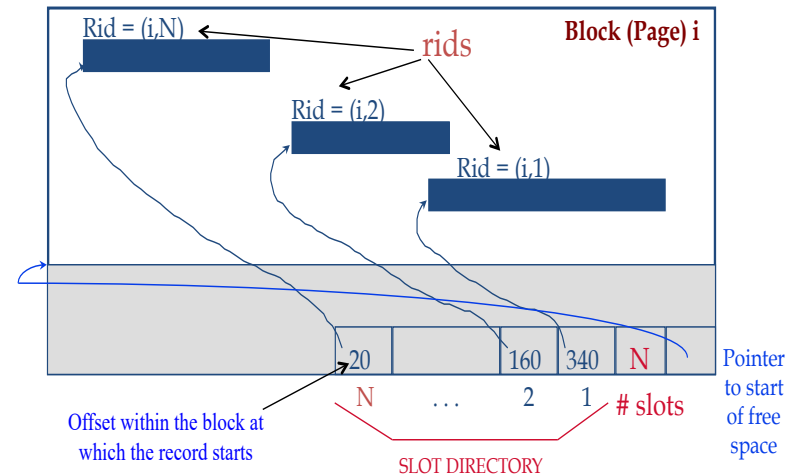
```
DROP TABLE IF EXISTS r;  
CREATE TABLE r (id INT PRIMARY KEY, val  
VARCHAR(6));  
INSERT INTO r VALUES  
(100, 'aaa'), (101, 'bbb'), (102, 'ccc');  
  
SELECT * FROM r;  
  
SELECT r.ctid, * from r;  
  
DELETE FROM r WHERE id = 100;  
  
SELECT r.ctid, * from r;
```

Summary

- Efficient design for packing records into disk blocks
- Works for both fixed- and variable-length records
- Record ids do not need to change

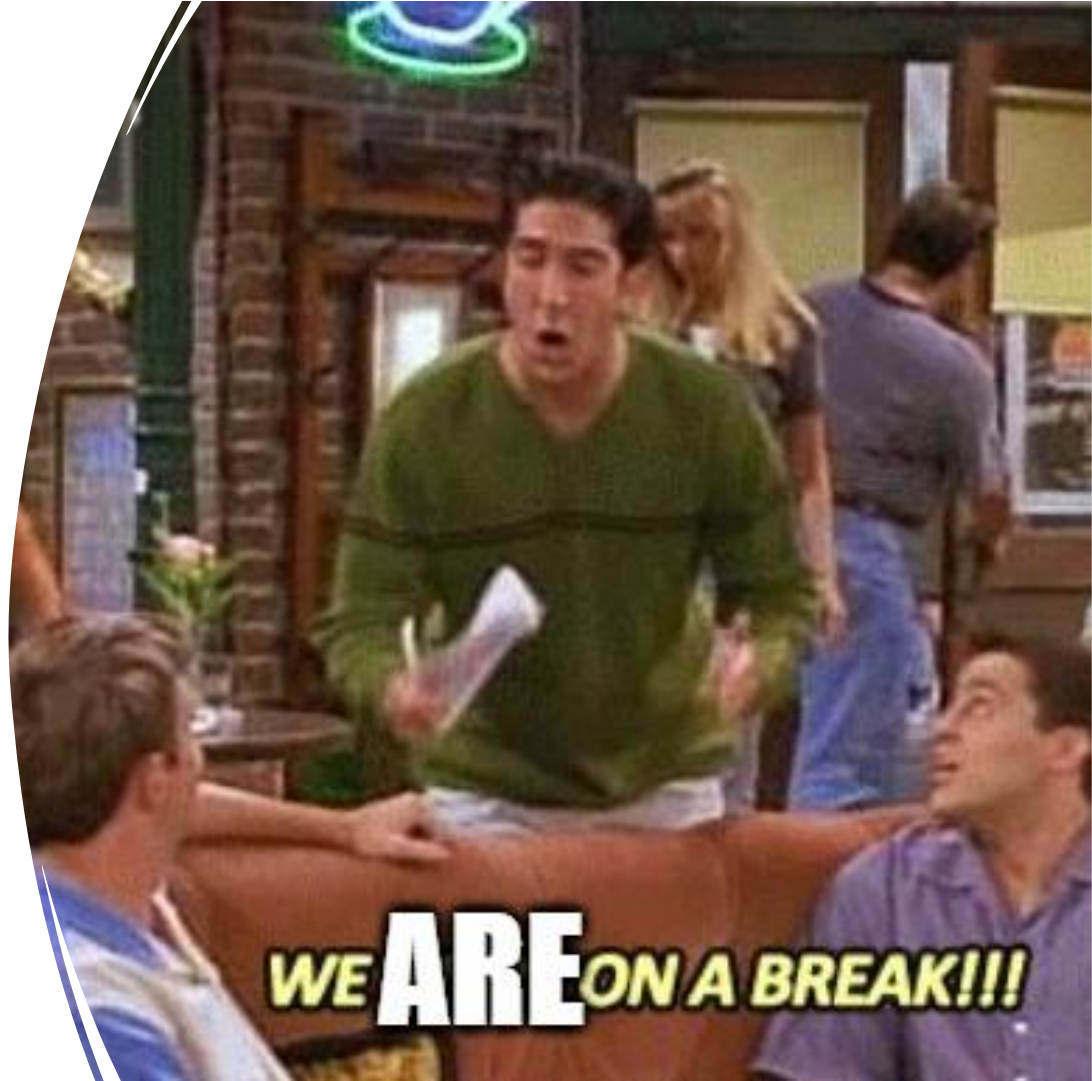
□ Record id (rid) = $\langle \text{Block id}, \text{slot \#} \rangle$

- Rid combines physical and logical Addresses
- I/U/D operations



Break

- Now we will take a few minutes break
- Make sure we have everything you need for the **Quiz**



Quiz 1

- **Reminders:**
 - **Closed book**
 - **Individual work**
 - **You have 15 minutes**

15:00

